PROJECT 01: University Scholarship Allocation

1. Problem Statement

Develop a program that checks scholarship applications against academic, attendance and family-income criteria, ranks eligible applicants, and applies clear tie-break rules.

2. Objectives

• Validate applicant marks, attendance and income data.
• Calculate aggregate marks and check every eligibility rule.
• Rank eligible applicants by aggregate, attendance, then lower family income.
• Display eligible rankings and rejected applicants' reasons.

3. Assumptions

• Marks are percentages on a 0–100 scale.
• Eligibility requires aggregate >= 75, every subject >= 60, attendance >= 75%, and annual family income <= Rs. 300,000.
• Tie-break order: higher aggregate, higher attendance, lower income, then applicant name.
• These thresholds are example rules and can be changed to match institutional policy.

4. Algorithm

1. Read the applicant records and policy thresholds.
2. Validate each applicant's marks, attendance and income.
3. Calculate the average of subject marks.
4. Check all eligibility conditions.
5. Store eligible applicants with their aggregate; record rejection reasons for others.
6. Sort eligible applicants using the stated tie-break rules.
7. Print the ranked list and rejection reasons.

5. Flowchart

START
↓
Input data and validate
↓
Process rules and calculations
↓
Check conditions / classify results
↓
Display result and END

6. Pseudocode

```
START
  SET minimum_aggregate = 75
  SET minimum_subject = 60
  SET minimum_attendance = 75
  SET maximum_income = 300000
  FOR each applicant
    CALCULATE aggregate = sum(marks) / number_of_subjects
    IF any input is outside its valid range
      REPORT invalid application
    ELSE IF all eligibility rules pass
      ADD applicant to eligible list
    ELSE
      REPORT rejection reason(s)
  SORT eligible by aggregate descending,
       attendance descending, income ascending, name ascending
  DISPLAY ranked eligible list
END
```

7. Python Code

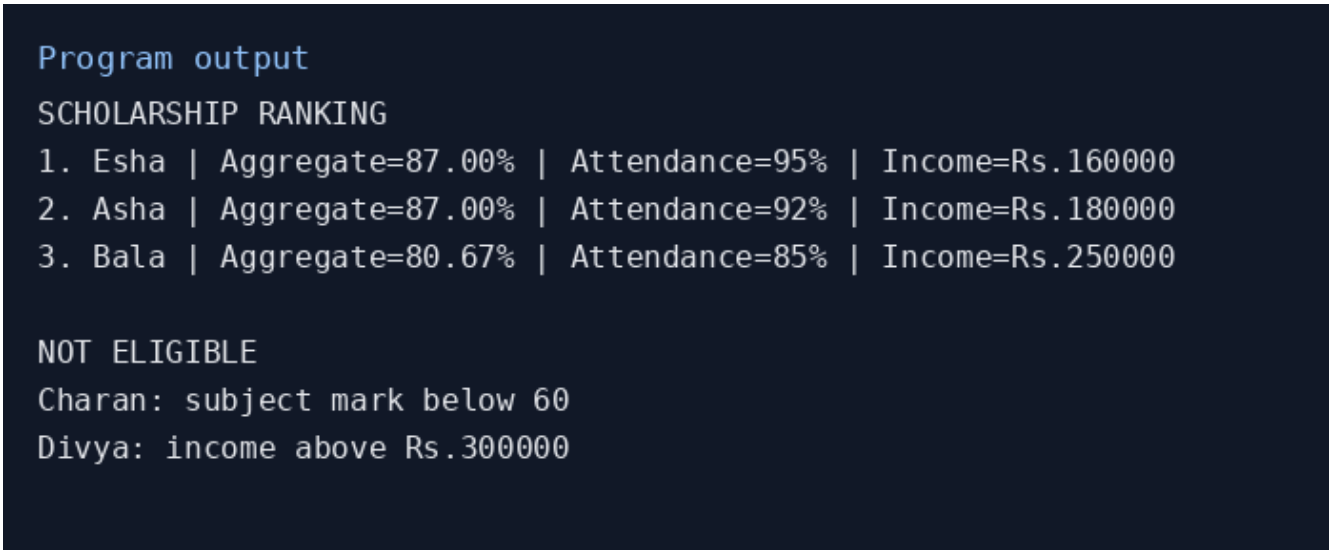
```
applicants = [
    {"name": "Asha", "marks": [88, 82, 91], "attendance": 92, "income": 180000},
    {"name": "Bala", "marks": [80, 78, 84], "attendance": 85, "income": 250000},
    {"name": "Charan", "marks": [92, 55, 90], "attendance": 95, "income": 150000},
    {"name": "Divya", "marks": [79, 81, 80], "attendance": 88, "income": 320000},
    {"name": "Esha", "marks": [88, 82, 91], "attendance": 95, "income": 160000},
]
eligible, rejected = [], []
for s in applicants:
    if not s["marks"] or any(m < 0 or m > 100 for m in s["marks"]) or not 0 <= s["attendance"]
<= 100 or s["income"] < 0:
        rejected.append((s["name"], "Invalid data"))
        continue
    avg = sum(s["marks"]) / len(s["marks"])
    reasons = []
    if avg < 75: reasons.append("aggregate below 75")
    if min(s["marks"]) < 60: reasons.append("subject mark below 60")
    if s["attendance"] < 75: reasons.append("attendance below 75%")
    if s["income"] > 300000: reasons.append("income above Rs.300000")
    if reasons:
        rejected.append((s["name"], ", ".join(reasons)))
    else:
        eligible.append({**s, "aggregate": avg})
eligible.sort(key=lambda s: (-s["aggregate"], -s["attendance"], s["income"], s["name"]))
print("SCHOLARSHIP RANKING")
for rank, s in enumerate(eligible, 1):
    print(f"{rank}. {s['name']} | Aggregate={s['aggregate']:.2f}% | Attendance={s[
'attendance']}% | Income=Rs.{s['income']}")
print("\nNOT ELIGIBLE")
for name, reason in rejected:
    print(f"{name}: {reason}")
```

8. Test Cases

Test	Input / condition	Expected result
Eligible applicant	Asha marks [88,82,91], attendance 92, income 180000	Included in ranked list
Subject minimum fails	Charan has a subject mark of 55	Rejected: subject mark below 60
Income limit fails	Divya income 320000	Rejected: income above Rs.300000
Tie-break	Asha and Esha have same aggregate; Esha has higher attendance and lower income	Esha ranks above Asha

9. Output Screenshot

The following screenshot-style image was generated from the captured output of the program execution.



Captured output

```
SCHOLARSHIP RANKING
1. Esha | Aggregate=87.00% | Attendance=95% | Income=Rs.160000
2. Asha | Aggregate=87.00% | Attendance=92% | Income=Rs.180000
3. Bala | Aggregate=80.67% | Attendance=85% | Income=Rs.250000

NOT ELIGIBLE
Charan: subject mark below 60
Divya: income above Rs.300000
```

10. Conclusion

The program automates eligibility checking, explains rejections, and ranks eligible applicants consistently using explicit tie-break rules.

Academic note: The program was executed with the sample data shown in the source code. Policies, rates, thresholds and exchange rates are illustrative and should be adjusted to the assignment's official requirements.